CONTENTS

1 General Setting **4**
- 1.1 Constraints and Environment . . . 4
- 1.2 System Overview . . . 4
- 1.3 Toolbox . . . 4
- 1.4 Motor DG01D-E . . . 5
- 1.5 Motor Driver L298P . . . 5
- 1.6 Calibration Result . . . 6
- 1.7 Motor Speed Measurement . . . 7
- 1.8 Discrete-time PID [10] . . . 7

2 Autonomous Line-Following Robot **10**
- 2.1 Project Objectives . . . 10
- 2.2 Robot Architecture . . . 10
- 2.3 Kinematic Model . . . 11
- 2.4 Robot Vision . . . 12
- 2.5 Navigation Logic . . . 12

3 Goalkeeper Robot **14**
- 3.1 Project Objectives . . . 14
  - 3.1.1 General Objective . . . 14
  - 3.1.2 Constraints and Environment . . . 14
  - 3.1.3 Approach . . . 14
- 3.2 Ball Detection . . . 16
  - 3.2.1 Color-Based Segmentation . . . 16
  - 3.2.2 Morphological Filtering and Contour Selection . . . 16
  - 3.2.3 Depth Estimation (Z) via Pinhole Projection . . . 16
  - 3.2.4 Lateral Offset Estimation (X) . . . 17
  - 3.2.5 Alternate Methods and Limitations . . . 17
- 3.3 Trajectory Estimation . . . 18
  - 3.3.1 Control Objective . . . 19
  - 3.3.2 Urgency Scaling . . . 19
  - 3.3.3 Control Law . . . 19
  - 3.3.4 Decision Loop . . . 20
- 3.4 Motor Actuation . . . 20
- 3.5 Theoretical Question: Posture and Performance Prediction Based on Robotic Motion Analysis . . . 21
  - 3.5.1 Objective and Methodology . . . 21
  - 3.5.2 Observations . . . 22
  - 3.5.3 Rule-Based Classifier for Ball Stopping . . . 22
  - 3.5.4 Regression Model for Error Estimation . . . 23

INTRODUCTION

The development of autonomous robots capable of perceiving and reacting to fast-moving objects has garnered significant attention across robotics, computer vision, and control systems. Applications range from industrial automation and autonomous driving to sports robotics and human–robot interaction. In this context, one particularly challenging task is the dynamic interception of a moving object — requiring fast sensing, accurate trajectory estimation, and precise actuation within tight time constraints.

This project originates from an early line-following robot initially designed to track a red-colored path using onboard vision. That platform served as a starting point to shift from path tracking to intercepting an object — specifically, designing a robotic goalkeeper capable of stopping a ball shot in its direction. While similar challenges have been addressed in advanced robotic platforms — such as humanoid soccer robots [2] or vision-based robotic arms in table tennis [3] — their core constraints remain relevant even in simplified scenarios.

The present work replicates this setup at a small scale using a lightweight, rectangular mobile robot equipped with a single fixed camera and driven by basic I2C-actuated motors. The robot is restricted to horizontal linear movement and must detect and stop a red ball shot from a short distance. The challenge lies in achieving this task with minimal computation and sensing, using only embedded image processing and feedback control logic.

Scientifically, the task integrates several foundational robotics problems:

- robust visual detection and filtering under constrained conditions,
- transformation of pixel measurements into real-world spatial coordinates,
- and real-time control logic based on evolving target dynamics.

While prior works have leveraged tools such as Kalman filtering [4], stereo triangulation [5], or deep learning prediction models [6], this project emphasizes a lightweight, interpretable approach. It uses HSV-based color segmentation and rule-based logic grounded in physical calibration and empirically measured motor response.

The remainder of this report details the design, implementation, and evaluation of the robot. The first part describes the general setting in which our project took place; the second addresses the main concepts of line-following while the last one describes the method used to build and evaluate the robot as a goalkeeper.

1 GENERAL SETTING

1.1 Constraints and Environment

- Robot dimensions: 20 cm (width) × 30 cm (length) × 8 cm (height)
- Rear wheels distances to robot's center of rotation: 11.5 cm.
- Front wheel distance to robot's center of rotation: 13.0 cm.
- Omni wheel radius: 2.8 cm.
- Camera height: 8 cm from ground
- Operating surface: flat, indoor lighting, static background

1.2 System Overview

The robot is based on a Raspberry Pi 4 with a PiCamera V2 wraper. Movement is achieved through DC motors controlled via an L298P H-bridge driver over I2C. The system processes images at 30 FPS and runs entirely onboard.

1.3 Toolbox

Several tools and libraries were employed throughout the development of the system:

- OpenCV: used for image processing and acquisition of meaningful data: barycenter of contour.
- LCCV: a C++ wrapper that enables real-time frame acquisition from the PiCamera using MMAL APIs, ensuring low-latency access to video streams.
- wiringPi: A library used to interface with the L298P motor driver over I2C, send PWM values, and issue directional commands to the motors.
- Processor: Raspberry Pi OS (64-bit) with embedded Linux environment.
- Measurement Tools: standard rulers and manual calibration setups were used to determine the camera constant and verify distance estimations. No high-speed camera or external sensors were used.

1.4 Motor DG01D-E

The DG01D-E motor features a built-in Hall effect sensor for detecting shaft rotation. When the sensor detects the south pole, it outputs a high logic level; when it detects the north pole, the output is low. This behavior is illustrated in the figure below. Shaft rotation direction is determined by analyzing the phase relationship between channels A and B:

1. If channel A detects a rising edge and channel B is high, which implies clockwise rotation.
2. If channel A detects a rising edge and channel B is low, which implies counterclockwise rotation.

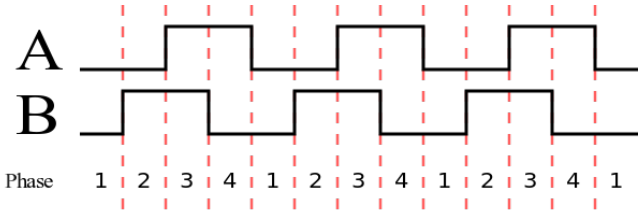


Figure 1: Quadrature encoder waveform with four phases per cycle, enabling direction detection. Source: Wikipedia.

In a `wiringPi`-based implementation, rising edges on channel A are detected using interrupts configured with `INT_EDGE_RISING`. The logic level on channel B is read via `digitalRead` to determine direction.

1.5 Motor Driver L298P

The motor is controlled via a Raspberry Pi using the L298P dual-channel motor driver. This driver supports up to 1 A per channel and operates with a DC power supply in the range of 6 V to 12 V. Communication with the Raspberry Pi is handled via the Grove I^2C interface, using the default address `0x0f`. When multiple drivers are used, their I^2C addresses can be configured using an address-selection jumper (also referred to as a wire hat). The module additionally provides a regulated 5 V output, which can be used to power external components. Motor speed is controlled by sending a PWM value to register `0x82`, while motor direction is set by writing to register `0xaa`, using the `wiringPi` library.

In this project, two motor drivers are used: one at address `0x0f` to control the left and right motors, and another at address `0x0d` to control the front motor.

1.6 Calibration Result

After performing multiple calibration tests, the following key parameters for the motor control system were determined:

Dead Zone PWM	40
Ticks per Revolution	144
Max No-load Wheel Velocity	120.0 TPS
Max Loaded Wheel Velocity	90.0 TPS
Velocity Measurement Resolution	±10.0 TPS

Due to the presence of a dead zone in motor, any PWM signal below a value of 40 results in zero effective shaft speed. Hence, one must take this nonlinearity into account. To mitigate this issue, we introduce a scaled PWM-to-speed mapping. Assuming linear behavior beyond the dead zone, the scaling factor is computed as:

$$\frac{255 - 40}{255} = 0.8431372549. \tag{1}$$

This gives that

$$\texttt{TPS}_{\text{ref}} = \frac{\texttt{PWM} - \texttt{DEAD_PWM}}{\texttt{SCALE_FACTOR}} \tag{2}$$

However, this linear model oversimplifies the real system, which is affected by motor nonlinearities and external disturbances (e.g., load variation, friction). Therefore, a closed-loop control strategy is required to accurately achieve the desired ticks per second for each motor.

Based on the calibration results, the following motion commands were defined as constants for the two rear motors controlled via the I^2C address `0x0f`:

Forward	`0x06`
Backward	`0x09`
Turn Left	`0x0a`
Turn Right	`0x05`

However, due to limited torque from the rear motors, the turning commands using only rear motors is proved to be ineffective. To address this, a third front motor was added to assist with turning maneuvers. For this configuration, the turn commands were determined for front motor of the driver's address `0x0d` as follows:

Turn Left	`0x06`
Turn Right	`0x09`

Since there is only one front wheel, the original direction commands for turning (`0x0a`, `0x05`) are now redundant.

1.7 Motor Speed Measurement

The speed measurement process proceeds as conventional:

1. Record the initial tick count, T_p.
2. Wait for a fixed time interval, Δt.
3. Record the final tick count, T_a.

The motor speed is then computed using:

$$\omega_m = \frac{T_a - T_p}{\Delta t} \tag{3}$$

To optimize performance, we avoid converting ticks per second (TPS) to revolutions per second by dividing by the number of ticks per revolution. This is because floating-point division is relatively expensive, typically requiring around 30 instructions, compared to only 1–3 instructions for most other floating-point operations. As a result, motor speed is represented directly in units of *ticks per second (TPS)*.

For low-level, instruction-constrained embedded systems, experimental results indicate that motor speed measurements should be performed *in-line*, rather than via a function call. Although the overhead of a function call is minimal—only a few nanoseconds as verified using Google Benchmark—minimizing latency is crucial for achieving scalability and responsiveness in real-time systems.

Based on testing, the minimum reliable time interval for speed measurement is 0.1 s. This interval is chosen to align with the PID controller's timing for computing derivative and integral terms, both of which are sensitive to temporal accuracy. Although shorter intervals could, in theory, improve responsiveness, they tend to introduce measurement noise and quantization artifacts due to encoder resolution and mechanical inertia. In practice, using a 0.1 s interval resulted in oscillations of approximately one tick—an acceptable compromise between responsiveness and stability.

1.8 Discrete-time PID [10]

In general, a PID control unit takes the following form in discrete-time formulation:

$$c[n+1] = k_p \cdot e[n] + k_i \cdot I[n+1] + k_d \cdot D[n+1] \tag{4}$$

where

1. $c[n+1]$ is the control signal sent to the actuator.

2. $e[n] := \omega_{\text{ref}} - \omega_{\text{mea}}$ is the error.
3. $I[n+1] := I[n] + e[n] \times \Delta t$ is the integral.
4. $D[n+1] := (e[n] - e[n-1])/\Delta t$ is the derivative.
5. k_p, k_i and k_d are respectively proportional gain, integral gain and derivative gain.

In our case, we control the motor by its speed. Therefore, the error is computed as the difference between reference angular velocity and measured one.

However, this control principle cannot be applied directly without modification. In particular, PID controllers are susceptible to *integral windup*. The integral term compensates for steady-state error, but when the actuator reaches its physical velocity limit, the system becomes nonlinear due to saturation. Despite this, the integrator continues to accumulate error, leading to overshoot and oscillations once the actuator recovers. This phenomenon prevents the motor speed to settle to desired velocity. This issue commonly arises during motor startup or when the wheels are obstructed. Computationally, the integral term is updated incrementally using a fixed sampling period of 0.1 s, but its update is applied only when the control output remains within the actuator bounds.

Additionally, the derivative term amplifies high-frequency noise, as it responds to rapid changes in the error signal. In systems with quantized or noisy sensors, this can lead to excessive control effort, actuator wear, and degraded performance. To mitigate this, a *low-pass filter* is typically applied to the derivative term to suppress noise-induced fluctuations.

In practice, the primary source of instability arises from measurement noise—particularly when one or two erroneous ticks are registered due to encoder bounce or sensor noise. These small variations can trigger abrupt control responses, making the system slow to settle. Applying a low-pass filter to the measured speed or to the derivative term effectively reduces this sensitivity and enhances overall stability.

A common and computationally efficient solution is to use a *first-order low-pass filter* with a single time constant. Its continuous-time transfer function is given by:

$$H(s) = \frac{1}{\tau s + 1}$$

where τ is the filter time constant.

In discrete-time, the filter is often implemented as an *exponential moving average*, whose recurrence relation is:

$$\tilde{e}_d[n] = \alpha \cdot e[n] + (1 - \alpha) \cdot \tilde{e}_d[n-1]$$

where:

- $e[k]$ is the current error.

- $\tilde{e}_d[k]$ is the filtered error.
- $\alpha = \cos(2\pi f_n) - 1 + \sqrt{\cos^2(2\pi f_n) - 4\cos(2\pi f_n) + 3}$ is the smoothing factor for normalized cutoff frequency $f_n := f_c/f_s$.
- Δt is the sampling interval.
- f_c is the cutoff frequency. It must always be less than half the sampling rate due to the Nyquist criterion.

The derivative is then calculated using finite differences of the filtered signal:

$$D[k] = \frac{\tilde{e}_d[n] - \tilde{e}_d[n-1]}{\Delta t} \tag{5}$$

To fine-tune the PID controller, we follow the conventional tuning steps found in standard control textbooks:

1. Begin with $K_i = 0$ and $K_d = 0$. Gradually increase K_p until sustained oscillations appear; denote this value as the ultimate gain K_u. Then reduce K_p to a stable value below K_u.
2. Adjust K_p incrementally. If the system response is sluggish, increase K_p; if oscillations or overshoot occur, decrease it. Repeat until a satisfactory proportional response is achieved.
3. Introduce K_d in small increments to dampen residual oscillations. Excessive derivative gain can amplify sensor noise and must be avoided.
4. Finally, increase K_i gradually to eliminate steady-state error while ensuring the integrator does not introduce instability.

Based on this procedure, the resulting PID parameters are as follows:

$$\begin{aligned} K_p &\cdots\cdots\cdots 0.3 \\ K_i &\cdots\cdots\cdots 0.005 \\ K_d &\cdots\cdots\cdots 0.005 \\ f_c &\cdots\cdots\cdots 4.0\,\text{Hz} \end{aligned}$$

This set of parameters yields a control response with a transient time of approximately 0.7 s, based on the criterion that stability is declared when the tick difference between successive measurements remains within 1 for five consecutive sampling intervals. The system typically reaches the desired value for the first time around 0.3 s.

2
AUTONOMOUS LINE-FOLLOWING ROBOT

2.1 Project Objectives

The goal of this project is to design and implement a miniature robotic platform capable of recognizing a line mix of red and blue color using camera and follow this line using a two rear motors and a front motor.

2.2 Robot Architecture

In the initial design, the robot followed a master-slave architecture, where the vision module acted as the master and directly commanded the motor control unit, acting as the slave. However, due to the fast-changing input and slow-to-stabilize nature of the motor's PID response, this centralized approach proved suboptimal. To address this, the system architecture was redesigned using a decentralized and parallel programming model. In the new setup, each control entity operates in its own dedicated thread, allowing the motors to make independent decisions based on local information and real-time feedback.

Additionally, as described in the navigation logic, normal operation runs under full PID control. However, when a sharp corner is detected and all motors are stalled, the system switches to an override mode where all motors are set to full throttle, and PID control is temporarily suspended. This behavior is coordinated through thread synchronization using standard C threading primitives. A shared synchronization flag, `THROTTLE_MUTEX`, is introduced to manage the transition between control modes. When the throttle override is active, all motor threads check the flag and suspend PID updates accordingly. Once the recovery phase is complete, the flag is cleared and normal PID operation resumes.

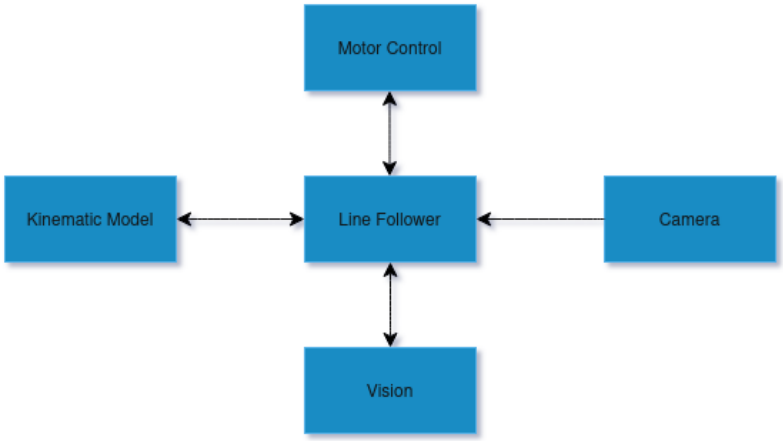


Figure 2: Multi-thread Programming architecture.

Robot multithread programming architecture is represented as in figure 2. We note from this diagram that line follower is a central unit which store all necessary variables. The frame data from camera is fed to this unit, then other units will access to this pool and process data respectively.

2.3 Kinematic Model

Due to the absence of torque data and the lack of sensors or estimators to infer it, a simplified control strategy based solely on velocity is adopted. This approach is referred to as *kinematic-based control*.

The reference angular velocities of the motors, denoted by $\omega_{r,1}$, $\omega_{r,2}$, and $\omega_{r,3}$, are derived from the desired translational velocity v and angular velocity ω of the robot. Assuming a wheel radius r, lateral wheel offset L_1, and longitudinal wheel offset L_2, the reference motor velocities are computed as:

$$\omega_{r,1} = \frac{1}{r}(v - L_1\omega) \cdot \kappa,$$
$$\omega_{r,2} = \frac{1}{r}(v + L_1\omega) \cdot \kappa,$$
$$\omega_{r,3} = \frac{1}{r} \cdot L_2\omega \cdot \kappa,$$

This model provides a straightforward mapping from robot-level commands (v, ω) to motor-level velocity references.

2.4 Robot Vision

The vision module is responsible for detecting red/blue lines and computing their barycenters for navigation tasks. To detect red and blue lines in the input image, color segmentation is performed in the HSV color space [7]. HSV is preferred over RGB because it separates chromatic content from illumination, making it more robust to changes in lighting conditions. For red detection, two separate HSV ranges are used:

1. $(0, 120, 70) : (10, 255, 255)$.
2. $(170, 120, 70) : (180, 255, 255)$.

For blue detection, a single HSV range is sufficient: $(100, 150, 50) : (140, 255, 255)$.

This section presents the implementation of a real-time monocular vision system for tracking a spherical object (ball) and controlling a mobile robotic platform accordingly. The system is based on a single RGB camera and performs depth estimation and lateral deviation computation using a geometric model of a pinhole camera.

The robot uses a monocular PiCamera and is controlled via an I2C motor driver interface. The camera continuously captures frames, and the ball is detected by color segmentation in the HSV space. The detected object's position and size are used to compute depth and deviation relative to the robot's axis.

The PiCamera is configured to a resolution of 640×480 pixels with a frame rate of 30 FPS. OpenCV is used for image processing, and the LCCV library facilitates camera capture.

2.5 Navigation Logic

The navigation module operates using a closed-loop control strategy, where feedback from the vision subsystem is used to compute motor commands in real time. For simplicity and efficiency, a PD controller is implemented to correct the robot's heading based on visual input:

$$\omega = K_p \cdot e + K_d \cdot \dot{e} \tag{6}$$

Here, ω is the rotational velocity command, e is the current visual error, and $\dot{e}$ is the rate of change of the error in sampling time $\Delta t = 0.1\,\text{s}$. The visual error e is computed as the horizontal distance between the barycenter of the detected line and the center of the image frame:

$$e = x - 320 \tag{7}$$

where x is the x-coordinate of the line's barycenter, and 320 is the horizontal midpoint of a 640-pixel-wide image.

The tuning process involves allowing the robot to follow the line autonomously and fully follow PD control logic. Experimentally, we found that:

$$k_p \cdots\cdots\cdots 0.005$$
$$k_d \cdots\cdots\cdots 0.01$$

While the PD controller performs well during normal turns, it fails in sharp turns where the robot can become stuck. These situations are identified by a large visual error—up to 172 pixels—and a measured motor velocity close to 0.0 TPS across all three motors. To recover from this condition, an override control logic is triggered: all three motors are driven at maximum speed in the direction indicated by the sign of the deviation.

This recovery action is applied for a fixed duration of 0.1 s, a value determined empirically to be sufficient for regaining motion. After this period, standard PID control is re-enabled, and the robot resumes normal operation.

3
GOALKEEPER ROBOT

3.1 Project Objectives

3.1.1 • General Objective

The goal is to make the robot intercept a rolling ball using only a top-mounted camera and lateral motion control. The robot must detect a red ball, predict its motion, and physically move to intercept it in real-time.

3.1.2 • Constraints and Environment

- Robot dimensions: 20 cm (width) × 30 cm (length) × 8 cm (height)
- Movement axis: lateral only (X-axis), range: 50 cm
- Camera height: 8 cm from ground
- Ball diameter: 4.6 cm
- Distance of shots: between 50 cm and 70 cm
- Operating surface: flat, indoor lighting, static background

3.1.3 • Approach

The development of the robotic goalkeeper was structured into three interdependent stages, each targeting a core functionality of the system: perception, trajectory reasoning, and motor control. This modular design allowed for incremental testing and optimization, while also enabling rapid adjustments to individual subsystems without compromising the global architecture.

1. Ball Detection and Localization. The first and most critical challenge was enabling the robot to perceive its environment through a top-mounted camera and accurately detect the ball. This required a vision pipeline capable of isolating the red ball from a potentially noisy or dynamic background, estimating its 2D position in the image, and then projecting that position into real-world coordinates relative to the robot. We approached this by building a color-based segmentation pipeline using HSV thresholds, followed by morphological filtering and contour analysis. Real-world coordinates were then obtained using a geometrical model based on the known size of the ball and its apparent size in pixels. Special care was taken to minimize latency and maintain consistent measurements across frames to support downstream

tasks.

2. Trajectory Estimation. The second development stage focused on interpreting the ball's motion based on consecutive detections. The initial strategy was to extract velocity vectors and fit a linear trajectory using two or more observed positions of the ball. This would allow the robot to compute where the ball was expected to cross the goal line and proactively move into place. However, this approach proved fragile in practice due to image noise, frame timing variability, and the inherently nonlinear behavior of the ball on the floor (e.g., slight curves, deceleration). After evaluating these issues through experimental trials, we pivoted toward a reactive strategy, where the robot continuously evaluates the ball's current position and adjusts its response dynamically, without relying on long-term prediction. This reactive control is more resilient to uncertainty and better suited to the short-range, high-variability environment in which the robot operates.

3. Motor Control and Real-Time Actuation. The final stage of development dealt with translating perceptual data into physical movement. Given the lateral position of the ball, the robot must decide on both the direction and speed of motion required to intercept it. A proportional control law was selected for its simplicity and fast response characteristics. The lateral offset of the ball was scaled by a manually tuned gain to compute the desired speed. To account for the urgency of the situation (i.e., how close the ball is to the goal line), the control signal was modulated by a nonlinear function of the estimated ball distance. This scaling increases responsiveness when the ball is near, without causing excessive movement when the ball is still far. Motor control was implemented through PWM signals issued over I2C to the L298P H-bridge driver, with additional logic to handle dead-zone compensation and direction encoding.

This structured, iterative approach ensured that each subsystem was tested and validated independently before full system integration, reducing complexity and improving overall reliability.

3.2 Ball Detection

3.2.1 • Color-Based Segmentation

We segment the red ball using HSV filtering. Because red wraps around the HSV hue range, we define two distinct intervals:

$$\begin{aligned}\text{Range 1: } & H \in [0, 10], \quad S \in [120, 255], \quad V \in [70, 255] \\ \text{Range 2: } & H \in [170, 180], \quad S \in [120, 255], \quad V \in [70, 255]\end{aligned}$$

3.2.2 • Morphological Filtering and Contour Selection

The binary mask obtained from HSV thresholding may contain small artifacts due to lighting or background noise. To address this, a morphological opening is applied: erosion removes small unwanted blobs, and dilation restores the shape of the main object.

Contours are then extracted using `cv::findContours`, configured to retrieve only external boundaries. Among the detected contours, the one with the largest area is selected, assuming the ball is the most prominent red object in view.

A minimum enclosing circle is fitted to this contour to estimate the ball's center (in pixels) and its apparent radius r. These values are later used for both distance estimation and lateral offset calculation.

3.2.3 • Depth Estimation (Z) via Pinhole Projection

Once the apparent radius of the ball is extracted from the image, the distance between the ball and the camera — referred to as the depth z — can be estimated using the pinhole camera model [8].

In this model, the relationship between the real diameter D of the object (in centimeters), its apparent diameter d in pixels, and its distance from the camera z is given by:

$$z = \frac{K \cdot D}{d}$$

Here, K is a calibration constant that combines the focal length of the camera (in pixels) and sensor scaling factors. In our case, $D = 4.6\,\text{cm}$ is the known diameter of the ball, and d is computed from the fitted enclosing circle as $d = 2r$.

To determine K, several images were captured of the ball at known distances. For each image, d was measured manually, and the formula was rearranged to solve for K. A best-fit

value was obtained using linear regression:

$$K = z \cdot \frac{d}{D}$$

yielding a final calibration constant of $K = 346.1$. We use the constant $K \times D = 1592.06$.

This method assumes a central perspective and minimal lens distortion, which is acceptable given the short camera height (8 cm) and the small working volume. For higher accuracy, lens distortion could be corrected via full camera calibration, but this was deemed unnecessary for the project's resolution and range.

3.2.4 • Lateral Offset Estimation (X)

Once the center of the ball is extracted from the image (in pixel coordinates), we compute its lateral offset from the robot's central axis. This offset represents how far left or right the ball is relative to the robot's current position, which is assumed to lie directly under the center of the image frame.

Let u be the x-coordinate of the ball's center in pixels, and let c_x be the center of the image along the x-axis. The pixel deviation from the center is given by $\Delta u = u - c_x$.

To convert this into a physical lateral distance x (in centimeters), we use the ratio between the apparent size of the ball and its known diameter. This yields the following equation:

$$x = \left(\frac{\Delta u}{d}\right) \cdot D$$

This formula is derived from the assumption that the width of the ball in pixels serves as a scaling factor between the image plane and real-world dimensions at the same depth. Since the ball is detected in the same frame as the center point u, both are at the same apparent scale.

This method avoids needing the camera's focal length or intrinsic calibration, relying instead on geometric ratios between pixel lengths and the known object size. It is valid under the assumption of a narrow field of view, a centered object, and a flat observation plane — all conditions satisfied in this setup.

3.2.5 • Alternate Methods and Limitations

Before settling on the final real-time reactive method, several other approaches were tested in an effort to predict the ball's trajectory in advance. Each had its own advantages but also significant drawbacks in our specific context.

Single Observation Prediction. The first idea was to use just two consecutive ball positions to estimate a linear trajectory. This approach fits a line $x(y)$ and computes the expected impact point when the ball reaches the robot's position ($y = 0$):

$$x = x_1 + \frac{x_2 - x_1}{y_2 - y_1}(0 - y_1)$$

Although conceptually simple, this method proved highly sensitive to noise. Small variations in pixel position, timing irregularities, or rolling imperfections caused large errors in the estimated slope, making predictions unreliable.

Multi-Point Linear Regression. A more robust alternative involved collecting multiple detections of the ball over time and applying linear regression to estimate a best-fit trajectory:

$$x = ay + b$$

This was computed by minimizing the total squared error over all observations. While it was more tolerant to noise and more stable overall, it required multiple frames to accumulate sufficient data, which introduced latency. Moreover, it still assumed that the ball moved in a straight line, which was often invalid due to surface irregularities or unintentional spin.

Limitations of Predictive Methods. Both predictive models also relied on precise synchronization between image acquisition and ball motion, which is difficult to achieve without hardware timing guarantees. Additionally, any acceleration or curvature in the ball's motion could not be accounted for, leading to errors at the moment of interception.

Final Choice: Reactive Estimation. Given these challenges, a reactive approach was chosen. Rather than predicting where the ball will be, the robot simply reacts to where it is now. The lateral position of the ball is computed at each frame, and the control system continuously adjusts the robot's movement based on this real-time information.

This decision trades long-range prediction for robustness and simplicity. For short-range applications with slow-to-moderate ball speed and frequent updates, reactive control provides more reliable behavior under uncertainty.

3.3 Trajectory Estimation

Given the limitations of predictive models discussed previously, the system implements a reactive estimation strategy, where the ball's current position in the frame is used to guide movement decisions in real time.

3.3.1 • Control Objective

The robot aims to align its position with the lateral position of the ball by the time the ball reaches the robot's plane ($y = 0$). Since exact trajectory prediction was deemed too sensitive and unreliable, we model the ball's observed position at each instant and compute a control action based on its current lateral deviation.

Let $x(t)$ denote the real-time lateral offset of the ball relative to the center of the robot (i.e., the x-distance in cm). The goal is to minimize this error dynamically as the ball approaches.

3.3.2 • Urgency Scaling

A key insight is that the robot must react faster as the ball gets closer to the goal line. To model this, we introduce an urgency factor that scales the control signal depending on the depth $z(t)$, which is the distance between the ball and the robot (computed via the pinhole model).

We define the urgency term as:

$$\text{urgency}(z) = \text{clamp}\left(\frac{z_{\max} - z}{z_{\max}},\, u_{\min},\, 1\right)$$

In practice:

- $z_{\max} = 100\,\text{cm}$ (maximum observed depth)
- $u_{\min} = 0.3$ (minimum urgency to avoid total inaction at far distances)

This non-linear scaling ensures that as the ball gets closer (i.e., $z \to 0$), the robot reacts more aggressively.

3.3.3 • Control Law

The robot's desired motor speed $v(t)$ is computed as:

$$v(t) = -K_p \cdot x(t) \cdot \text{urgency}(z(t))$$

where:

- $x(t)$ is the lateral error (in cm)
- $z(t)$ is the estimated depth to the ball (in cm)
- K_p is the proportional gain, tuned experimentally ($K_p = 100$)

This equation ensures that the robot moves faster when the ball is both far off-center and close to the robot, and more slowly when it is nearly aligned or far away. The sign of the control signal encodes the direction: positive for left, negative for right.

3.3.4 • Decision Loop

At each time step:

1. A new frame is captured.
2. The ball is detected, and its center is extracted.
3. Depth $z(t)$ and lateral offset $x(t)$ are estimated.
4. Speed $v(t)$ is calculated using the control law.
5. Transmit the speed and direction to the motor driver to initiate movement.

This structure avoids the pitfalls of predictive estimation while maintaining responsive behavior suitable for short-range, real-time interception tasks.

3.4 Motor Actuation

The robot executes a closed-loop control cycle at a regular interval, typically aligned with the camera frame rate. The logic relies exclusively on the current frame, without memory or prediction, making it highly reactive and robust to noise.

The process is outlined below:

```
Loop:

    1. Capture image frame from camera
    2. Detect red ball and extract (x, z) position:
        - x: lateral offset from robot center (in cm)
        - z: estimated distance to robot (in cm)
    3. Compute urgency factor from depth z
    4. Compute speed = -Kp * x * urgency
    5. Determine direction (left or right)
    6. Apply dead-zone compensation
    7. Send direction and speed to motor driver over I2C
```

Although the control system is open-loop in terms of position (no encoders or feedback), the precision of visual tracking and the short movement range (50 cm) allow for sufficient reliability in real-world trials.

3.5 Theoretical Question: Posture and Performance Prediction Based on Robotic Motion Analysis

3.5.1 • Objective and Methodology

The aim of this analysis is to determine whether the initial parameters — defined by the ball's initial position, angle of shooting, and estimated speed — can be used to predict:

1. Whether the robot successfully stops the ball (classification),
2. The lateral positioning error between the ball and the robot at the moment of impact (regression).

We rely on a dataset of controlled trials, each annotated with input features and outcome measures. A simple rule-based classifier is used for task (1), and a multivariate linear regression model is used for task (2).

Each trial is characterized by the following predictive variables:

- **Initial distance** to the ball d [cm],
- **Angle of shooting** θ [degrees],
- **Estimated ball speed** v [cm/s], computed as:

$$v = \frac{\sqrt{d^2 + x^2}}{t}$$

where x is the lateral offset (final coordinate at $y = 0$) and t is the travel time in seconds.

The two predicted outputs are:

- **Success label**: whether the ball was stopped, based on the condition that the error from the robot center was less than 10 cm.
- **Final error** E: the measured deviation from the robot's center [cm].

3.5.2 • Observations

Table 1: Experimental Trials: Ball Trajectory and Outcome

Initial Distance (cm)	Shooting Angle (°)	Ball Speed (cm/s)	Ball Stopped	Final Error (cm)
70	-20	72.8	No	14
70	-5	70.7	Yes	0
70	-40	76.2	Yes	5
70	40	72.4	No	26
70	20	71.3	No	17
70	40	72.5	No	24
70	15	70.9	Yes	5
70	10	70.6	Yes	4
60	25	66.2	No	21
60	10	48.1	Yes	3
60	5	38.8	Yes	0
60	-15	48.6	Yes	2
60	-15	33.4	Yes	2
60	-30	45.3	Yes	5
50	15	39.8	Yes	3
50	25	44.4	No	18
50	-20	45.6	Yes	3
50	-40	51.0	Yes	5

3.5.3 • Rule-Based Classifier for Ball Stopping

A simple rule-based model[12] was designed using thresholding on three input features: the angle of shooting, the estimated ball speed, and the initial distance from the robot. The rule is formulated as follows:

$$\textit{If} \quad |\theta| \leq 20°,\ v \leq 50\,\text{cm/s},\ \textit{and}\ d \geq 60\,\text{cm} \Rightarrow \text{ball is likely to be stopped.}$$

The rule was empirically tuned from observation of the dataset and reflects the robot's mechanical limitations: short-range high-speed shots are significantly harder to intercept, even at low angles. Longer distances provide more time for detection and actuation.

We evaluated this rule on the complete dataset, comparing its predictions to the actual outcomes of the trials. From this evaluation, we computed the following performance metrics:

- **Accuracy**:
$$A = \frac{\#\text{ correct predictions}}{\text{total trials}} = \frac{10}{18} \approx 55.6\%$$
- **False positives**: 0
- **False negatives**: 8

This rule-based approach, though simple, provided an interpretable framework to link robot posture and environmental conditions to interception performance. It offers an efficient method to anticipate success scenarios.

3.5.4 • Regression Model for Error Estimation

To predict the final lateral error E [cm], we fitted a multivariate linear regression model based on the robot's initial posture and ball characteristics:

$$\hat{E} = \alpha_0 + \alpha_1 \cdot d + \alpha_2 \cdot \theta + \alpha_3 \cdot v$$

Using the dataset of 18 experimental trials, we computed the coefficients via least squares [9]:

$$\hat{E} = -15.82 + 0.082 \cdot d + 0.205 \cdot \theta + 0.227 \cdot v$$

The model was evaluated on the same dataset using standard error metrics:

- Mean Absolute Error (MAE): $\approx 4.42\,\text{cm}$
- Root Mean Squared Error (RMSE): $\approx 5.46\,\text{cm}$
- Coefficient of determination R^2: ≈ 0.68

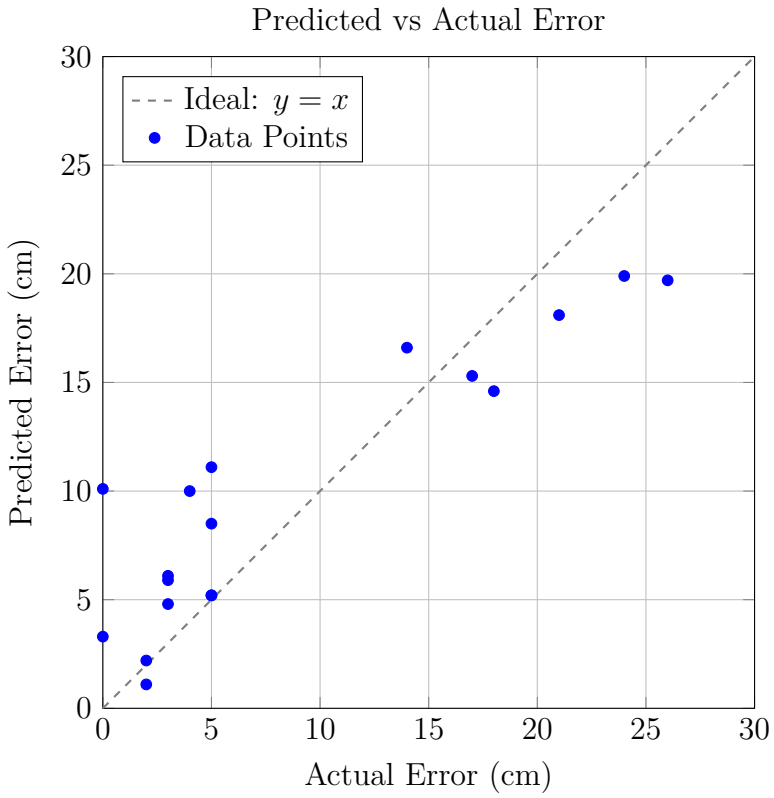


Figure 3: Predicted vs. Actual Lateral Error from Regression Model

This model indicates a clear linear relationship between motion characteristics and error. Both speed and shooting angle are strong predictors of poor interception, while initial distance contributes moderately. The regression can serve as a predictive tool or diagnostic measure to evaluate robot posture adequacy during operation.

Initial results indicate that high angles and fast ball speeds are strongly associated with failure. In contrast, low-speed, low-angle shots were successfully stopped in nearly all cases. Among the three features, the estimated ball speed showed the strongest correlation with final error, followed by angle.

CONCLUSION

Line Follower Robot

The line-following program achieves reliable path tracking with good precision, though the resulting navigation is not entirely smooth. A noteworthy emergent behavior is observed when the robot encounters a sharp turn: it first reverses slightly to realign its pose into a turn-favorable configuration before executing the turn with increased motor effort. This behavior reflects the strength of a model-based control approach, which would be difficult to achieve using purely ad hoc logic. While this result should not be overstated, it provides evidence that the control logic is well-structured and effective.

During development, OpenMP was tested to parallelize the image processing pipeline. However, performance gains were negligible, with processing time averaging around 20.5 ms per frame, indicating the vision module is already near optimal. The motor response exhibited no abrupt changes in wheels' direction, confirming that the PID controllers are well-tuned for both stability and responsiveness.

For future work, we plan to integrate a machine learning-based navigation model capable of handling branched line paths, allowing the robot to recognize previously visited lines and backtrack from dead ends to the nearest intersection. Additionally, an event-driven control mechanism will be explored to enable manual override when no line is detected.

One limitation of the current system is the use of multithreading without formal verification. Although practical testing shows stable behavior, there is no theoretical guarantee regarding thread safety or deadline satisfaction. Future iterations may benefit from a more rigorous real-time system analysis.

Goalkeeper Robot

Due to time constraints, the development of the goalkeeper robot was limited to simple electronic devices, and we have not yet covered the power electronics part, which is responsible for the time reactivity of the robot. By a rough estimation, a general goalkeeper has to have a lateral velocity of about $2\,\mathrm{m\,s^{-1}}$ in order to intercept a fast-moving ball travelling at $6\,\mathrm{m\,s^{-1}}$ from a distance of 70 cm. This requires motor systems capable of delivering over 10 RPS with sufficient torque, and power electronics that can respond within 30 ms or less, matching the reaction window determined by the vision system's framerate and processing time.

Given these timing requirements, a microcontroller is indispensable for handling real-time operations with deterministic timing guarantees. While the Raspberry Pi handles vision and

strategy, it lacks real-time capabilities due to its non-deterministic Linux environment. Therefore, a dedicated MCU such as a Teensy 4.1 or STM32F4 is necessary to manage precise PWM generation, motor control loops, and low-latency response routines. Such MCUs can operate reliably at control loop frequencies exceeding 1 kHz, allowing the system to issue motor corrections in under 1 ms — a requirement if one hopes to approach human-level reflexes in robotic interception.

In terms of mechanical design, switching to a linear rail system offers a clear advantage over wheeled platforms, especially in situations where fast and precise side-to-side movement is critical—like in robotic goalkeeping. Unlike wheels, which can slip or wobble under rapid motion, a belt-driven or lead-screw rail setup keeps the robot firmly on track and eliminates lateral drift. Thanks to its rigid structure and low-friction guidance, the rail allows the goalkeeper to accelerate and stop almost instantly, making quick, controlled dives across the goal not only possible, but reliably repeatable. However, this is the story of future project.

REFERENCES

[1] Karen Sparck Jones. 1972. A Statistical Interpretation Of Term Specificity And its Application In Rretrieval. *Journal of Documentation.*Vol. 28 No. 1, pp. 11-21

[2] RoboCup Federation. "RoboCup Soccer League." *robocup.org*, 2023.

[3] Zhang, Y. et al. "Deep Imitation Learning for High-Speed Ball Hitting in Robotic Table Tennis." *RA-L*, 2018.

[4] Kim, H. et al. "Real-time ball tracking using Kalman filter in soccer robot system." *ICRA*, 2004.

[5] Liu, Y. et al. "3D ball localization for robot soccer using stereo vision." *IJARS*, 2012.

[6] Chen, M. et al. "End-to-end learning of goal-directed actions from visual input in robotics." *CoRL*, 2019.

[7] Gonzalez, R. C., & Woods, R. E. (2002). *Digital Image Processing*. Prentice Hall.

[8] Hartley, R., & Zisserman, A. (2003). *Multiple View Geometry in Computer Vision*. Cambridge University Press.

[9] Strang, G. (2009). *Introduction to Linear Algebra* (4th ed.). Wellesley-Cambridge Press.

[10] Åström, K. J., & Hägglund, T. (1995). *PID Controllers: Theory, Design, and Tuning* (2nd ed.). ISA.

[11] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **22**(11), 1330–1334.

[12] Russell, S. J., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach* (3rd ed.). Pearson Education.